Mathematical Explanation of Trip-Planner-Crew

Authors: Ph.D. N.Momtselidze with AI assistance

Overview

The Trip-Planner-Crew is a multi-agent AI system built using CrewAI that coordinates three specialized agents to generate comprehensive travel itineraries. This document explains the mathematical foundations and algorithms underlying the system.

System Architecture

Multi-Agent Coordination Model

The system employs a **distributed multi-agent system** with three specialized agents:

1. **City Selection Agent** (A_1)
2. **Local Expert Agent** (A_2)
3. **Travel Concierge Agent** (A_3)

The coordination follows a **sequential task execution pattern**:

$$T_1 \rightarrow T_2 \rightarrow T_3$$

Where: - T_1 = identify_task (executed by A_1) - T_2 = gather_task (executed by A_2) - T_3 = plan_task (executed by A_3)

Task Dependency Graph

The task structure forms a **Directed Acyclic Graph (DAG)**:

$$G = (V, E)$$
$$V = \{T_1, T_2, T_3\}$$
$$E = \{(T_1, T_2), (T_2, T_3)\}$$

This ensures that: - Task T_2 cannot start until T_1 completes - Task T_3 cannot start until T_2 completes - No cycles exist (acyclic property guarantees termination)

Mathematical Components

1. Cost Optimization and Budget Calculation

The Travel Concierge agent uses the CalculatorTools for budget optimization:

Budget Model:

$$B_{total} = B_{accommodation} + B_{transport} + B_{food} + B_{miscellaneous}$$

Where: - $B_{accommodation} = \Sigma$ (price_i × nights) for i ∈ accommodations - $B_{transport} = \Sigma$ (cost_j) for j ∈ transport_modes - $B_{food} = \Sigma$ (daily_food_cost × days) - $B_{miscellaneous}$ = buffer_margin × B_{total}

The calculator tool evaluates mathematical expressions using Python's `eval()` function:

```
        def calculate(operation):
            return eval(operation)
```

This allows for complex budget calculations such as: - `200*7` (hotel cost × 7 nights) - `5000/2*10` (currency conversion and multiplication)

2. Information Retrieval and Ranking

The SearchTools implements a **top-k retrieval algorithm**:

```
R = search(query)
S = sort(R, relevance_score)
return S[0:k]  where k = 4
```

Parameters: - k = 4 (top_result_to_return) - Query space Q ⊆ ℝ^n (n-dimensional feature space) - Relevance function: rel: Q × D → [0,1] where D is the document space

Algorithm Complexity: - Time: O(n log n) for sorting n results - Space: O(k) for storing top k results

3. Content Chunking Algorithm

The BrowserTools implements a **fixed-size chunking algorithm** for processing large web content:

```
C = content
chunk_size = 8000 characters
n_chunks = ⌈|C| / chunk_size⌉

chunks = [C[i:chunk_size] for i in range(0, |C|, chunk_size)]
```

Mathematical Properties: - For content of length L, number of chunks = ⌈L/8000⌉ - Chunk i spans indices: [i×8000, min((i+1)×8000, L)) - Union of all chunks = C (partition property) - Intersection of any two chunks = ∅ (disjoint property)

Complexity: - Time: O(L) to partition content - Space: O(L) to store all chunks

4. Multi-Agent Task Scheduling

The CrewAI framework uses a **task scheduling algorithm**:

Task Execution Model:

```
For each task T in [T₁, T₂, T₃]:
    while not T.is_complete():
        T.execute()
        if T.requires_tool():
            tool_result = T.tool(T.current_state)
            T.update_state(tool_result)
```

State Transition Function:

```
δ: S × A → S'
```

Where: - S = current state of the task - A = action (tool execution or LLM generation) - S’ = new state after action

Dependency Satisfaction:

```
ready(T_i) = ∀ (T_j, T_i) ∈ E: T_j.is_complete()
```

5. Agent Selection Logic

The system uses **conditional logic** for LLM provider selection:

```
LLM = f(API_keys) where:
```

```
f(keys) =
    'gemini/gemini-2.0-flash' if GOOGLE_API_KEY ∈ keys
    'gpt-3.5-turbo' if OPENAI_API_KEY ∈ keys
    None otherwise
```

This is a **piecewise function** defined over the API key space.

6. Fallback Planning Algorithm

When LLM is unavailable, the system uses a **deterministic fallback plan**:

```
Fallback(origin, city, date, interests) → Plan
```

The fallback plan is a **static mapping**:

```
P_fallback = {
    day_1: "Arrival and historic district walk",
    day_2: "University area exploration",
    day_3: "Academic neighborhoods and libraries",
    day_4: "Accommodation and commute analysis",
    day_5: "Museums and cultural landmarks",
    day_6: "Student services and practical essentials",
    day_7: "Flexible follow-up day"
}
```

This represents a **constant-time lookup** O(1) for the plan structure.

Probability and Uncertainty

API Connection Verification

The `verify_gemini_connection` function represents a **probabilistic success model**:

```
P(success) = P(API_key_valid) × P(network_available) ×
P(model_responsive)
```

The verification is modeled as a **Bernoulli trial**:

```
X ~ Bernoulli(p) where:
X = 1 if connection successful
X = 0 if connection fails
```

Error Handling

The system uses **exception handling** to manage uncertainty:

```
try:
    result = crew.kickoff()
except Exception as error:
    return fallback_plan(...)
```

This can be modeled as:

```
Result = {
    crew.kickoff() with probability p_success
    fallback_plan() with probability (1 - p_success)
}
```

Information Flow

Data Pipeline

The information flow through the system:

```
Input: I = {origin, cities, date_range, interests}

Output: O = TripCrew.run(I)
```

Where:
$O = A_3(T_3(A_2(T_2(A_1(T_1(I))))))$

This represents a **composition of functions**:

$O = f_3 \circ f_2 \circ f_1(I)$

Tool Composition

Each agent has access to a set of tools:

$Tools(A_1) = \{search, browser\}$
$Tools(A_2) = \{search, browser\}$
$Tools(A_3) = \{search, browser, calculator\}$

The tool execution follows a **functional composition** pattern:

$tool_result = tool_1 \circ tool_2 \circ ... \circ tool_n(input)$

Algorithmic Complexity Analysis

Overall System Complexity

Let: - n = number of search results per query - k = top results returned (k = 4) - L = total content length scraped - c = chunk size (c = 8000) - m = number of chunks = $\lceil L/c \rceil$

Time Complexity: - Search: O(n log n) for sorting - Scraping: O(L) for content retrieval - Chunking: O(L) for partitioning - Summarization: O(m × t) where t = LLM inference time per chunk - Total: O(n log n + L + m × t)

Space Complexity: - O(k) for search results - O(L) for scraped content - O(L) for chunks - Total: O(L)

Mathematical Properties

Correctness

Invariant: Task dependencies are always satisfied before execution - Proof: The DAG structure and sequential execution ensure T_2 only runs after T_1 completes, and T_3 only after T_2 completes.

Termination: The system always terminates - Proof: The task graph is finite and acyclic, with no loops. Each task either succeeds or triggers fallback, both of which are terminal states.

Optimality

The system aims for **Pareto optimality** in trip planning: - Maximize: attractions visited, cultural experiences, practical utility - Minimize: cost, travel time, complexity

The multi-agent approach allows **specialization**: - A_1 optimizes city selection - A_2 optimizes local knowledge - A_3 optimizes itinerary and budget

Conclusion

The Trip-Planner-Crew system combines several mathematical concepts:

1. **Graph Theory** - DAG-based task dependencies
2. **Optimization** - Budget and itinerary optimization
3. **Information Retrieval** - Top-k search algorithms

4. **Algorithm Design** - Chunking, sorting, and sequential processing
5. **Probability** - Success/failure modeling for API connections
6. **Functional Composition** - Multi-agent information flow

The mathematical foundation ensures the system is correct, terminates, and provides optimal trip planning solutions within the constraints of available data and computational resources.